

Chapter 12 Boolean Algebra

Chapter Summary

- Boolean Functions
- Representing Boolean Functions
- Logic Gates
- Minimization of Circuits

Section 12.1 Boolean Functions

Section Summary

- Introduction to Boolean Algebra
- Boolean Expressions and Boolean Functions
- Identities of Boolean Algebra
- Duality
- The abstract definition of a Boolean Algebra

Introduction to Boolean Algebra

Boolean algebra has rules for working with element with binary set $\{0, 1\}$ together with operator Boolean sum(+), Boolean product($\cdot$), complement($\bar{}$)

These operators are defined by:

- Boolean sum:(similar to $\vee$)
 $1 + 1 = 1, 1 + 0 = 1, 0 + 1 = 1, 0 + 0 = 0$
- Boolean product(similar to $\wedge$):
 $1 \cdot 1 = 1, 1 \cdot 0 = 0, 0 \cdot 1 = 0, 0 \cdot 0 = 0$
- Complement:
 $\bar{0} = 1, \bar{1} = 0$

Boolean Expressions and Boolean Functions

Definition:

Let $B = \{0, 1\}$, then $B^n = \{(x_1, x_2, \dots, x_n) | x_i \in B\}$ is the set of all possible n -tuples of 0s and 1s. The variable x is called a Boolean variable if it assumes value only from B . (only 0 or 1) A function from B^n to B is called a Boolean function of degree n .

Example, $F(x, y) = x$, $F(x, y, z) = xy + \bar{z}$.

example table:

TABLE 1		
x	y	$F(x, y)$
1	1	1
1	0	0
0	1	0
0	0	1

Boolean Expressions and Boolean Functions

Definition:

Boolean functions F and G of n variables are **equal** if and only if $F(b_1, \dots, b_n) = G(b_1, \dots, b_n)$ whenever $b_1, \dots, b_n \in B$. Two different Boolean expressions that represent the same function are **equivalent**.

Definition:

The **complement** of Boolean function F is $\bar{F}$, where $\bar{F}(x_1, \dots, x_n) = F(x_1, \dots, x_n)$.

Definition:

Boolean sum and Boolean product of two Boolean functions F and G are defined by

$$\begin{aligned}(F + G)(x_1, \dots, x_n) &= F(x_1, \dots, x_n) + G(x_1, \dots, x_n) \\ (FG)(x_1, \dots, x_n) &= F(x_1, \dots, x_n)G(x_1, \dots, x_n)\end{aligned}$$

Example:

The number of different Boolean functions of degree n is 2^{2^n} . (2^n different n -tuples, each tuple corresponding 2 situations of output)

Identities of Boolean Algebra

TABLE 5 Boolean Identities.	
<i>Identity</i>	<i>Name</i>
$\overline{\overline{x}} = x$	Law of the double complement
$x + x = x$ $x \cdot x = x$	Idempotent laws
$x + 0 = x$ $x \cdot 1 = x$	Identity laws
$x + 1 = 1$ $x \cdot 0 = 0$	Domination laws
$x + y = y + x$ $xy = yx$	Commutative laws
$x + (y + z) = (x + y) + z$ $x(yz) = (xy)z$	Associative laws
$x + yz = (x + y)(x + z)$ $x(y + z) = xy + xz$	Distributive laws
$\overline{(xy)} = \overline{x} + \overline{y}$ $\overline{(x + y)} = \overline{x} \overline{y}$	De Morgan's laws
$x + xy = x$ $x(x + y) = x$	Absorption laws
$x + \overline{x} = 1$	Unit property
$x\overline{x} = 0$	Zero property

Example: to show a expression is valid, draw the table of Boolean function.

Formal Definition of a Boolean Algebra

Definition:

A **Boolean algebra** is a set B with two binary operations $\vee$ and $\wedge$, element 0 and 1, a unary operation $\neg$ such that for all $x, y, z \in B$ satisfied identity law, complement law, associative law, commutative law, distributive law.

The set regarding set theory rather than propositional variable can also represent a Boolean algebra.

Section 12.2 Representing Boolean Functions

Section Summary

- Sum-of-Products Expansions
- Functional Completeness

Sum-of-Product Expansion

Example: Find Boolean expression represent function F and G .

TABLE 1				
x	y	z	F	G
1	1	1	0	0
1	1	0	0	1
1	0	1	1	0
1	0	0	0	0
0	1	1	0	0
0	1	0	0	1
0	0	1	0	0
0	0	0	0	0

General principle is that each combination of values of the variables for which the functions have the value 1 requires a term in the Boolean sum that is the Boolean product of the variables or their complements.

Definition:

A **literal** is a Boolean variable or its complement. A **min term** of the Boolean variable $x_1, \dots, x_n$ is a Boolean product $y_1, \dots, y_n$ where $y_i = x_i$ or $y_i = \bar{x}_i$. Hence, a min term is a **product of n literals** with one literal for each variable.

The min term $y_1, \dots, y_n$ has value 1 if and only if each $x_i = 1$ for $y_i = x_i$, $x_i = 0$ for $y_i = \bar{x}_i$.

Definition:

The sum of min terms that represents the function is called **sum-of-products expansion** or **disjunctive normal form** of the Boolean function.

Example: Find the sum-of-products expansion for $F(x, y, z) = (x + y)\bar{z}$.

- Method 1: Using a table
- Method 2: Using Boolean Identities.

$$\begin{aligned}
 F(x, y, z) &= (x + y)\bar{z} \\
 &= x\bar{z} + y\bar{z} \\
 &= x1\bar{z} + 1y\bar{z} \\
 &= x(y + \bar{y})\bar{z} + (x + \bar{x})y\bar{z} \\
 &= xy\bar{z} + x\bar{y}\bar{z} + xy\bar{z} + \bar{x}y\bar{z} \\
 &= xy\bar{z} + x\bar{y}\bar{z} + \bar{x}y\bar{z}
 \end{aligned}$$

Functional Completeness

Definition:

Because every Boolean function can be represented using Boolean operator set $\{\cdot, +, \neg\}$, which is **functionally complete**.

- $\{\cdot, \neg\}$ is functionally complete since $x + y = \bar{x}\bar{y}$
- $\{+, \neg\}$ is functionally complete since $xy = \bar{\bar{x} + \bar{y}}$
- nand operator, denoted by $|$, defined by $1|1 = 0$, $1|0 = 0|1 = 0|0 = 1$ is functionally complete.
- nor operator, denoted by $\downarrow$, defined by $0 \downarrow 0 = 1$, $1 \downarrow 0 = 0 \downarrow 1 = 1 \downarrow 1 = 0$, is functionally complete.

Section 12.3 Logic Gates

Section Summary

- Logic Gates
- Combination of Gates
- Example of Circuits

Logic Gates

Gates take as input of two or more Boolean variables and produce one or more bit as output. And **inverters** take the value of Boolean variable as input and produce its complement as output.



(a) Inverter



(b) OR gate



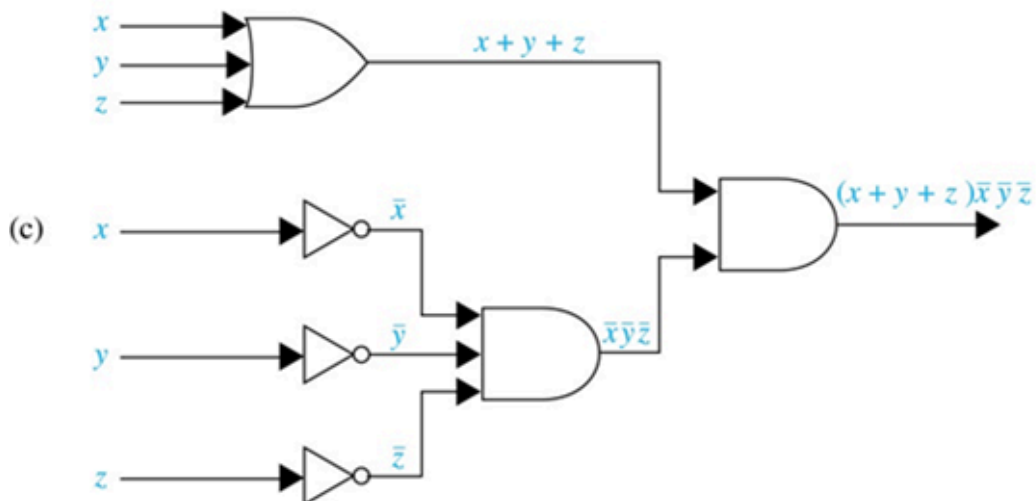
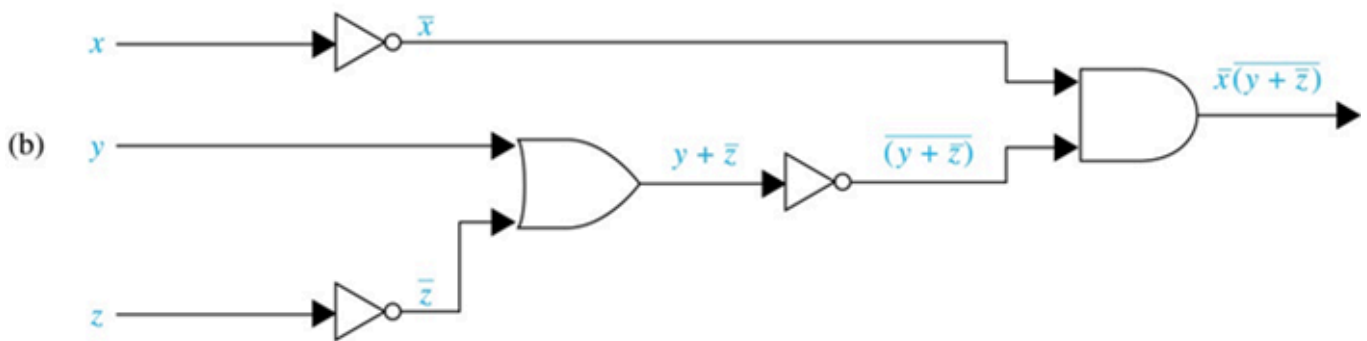
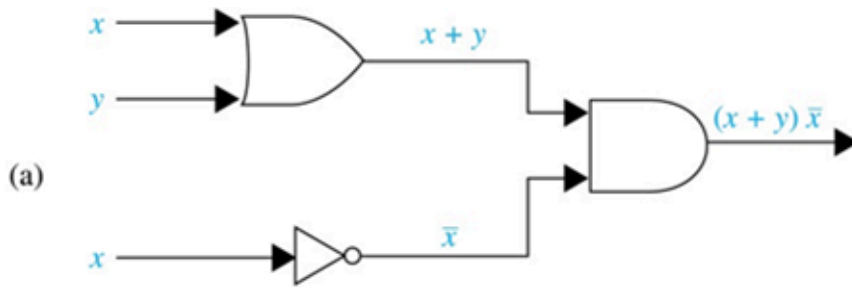
(c) AND gate

Combinations of Gates

Combinational circuits can be constructed using a combination inverters and gates. Gates can share input and the output may be the input of another gate.

Example:

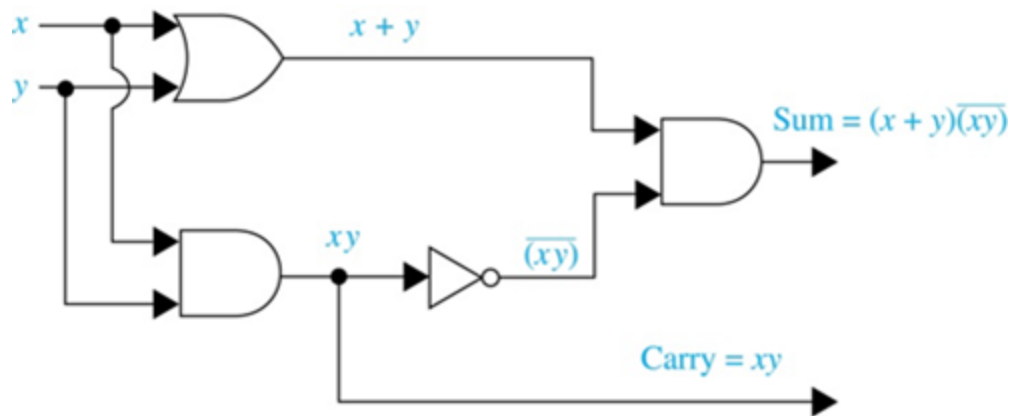
- $(x + y)\bar{x}$
- $\bar{x}(y + \bar{z})$
- $(x + y + z)(\bar{x}\bar{y}\bar{z})$



Adders

Half Adder

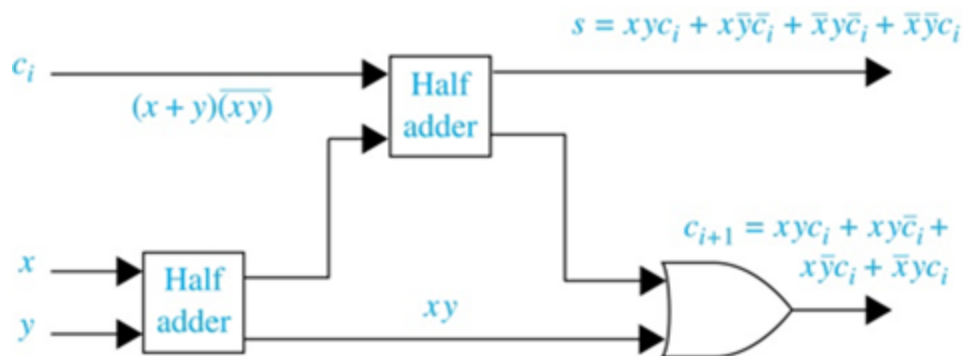
Add two bits and output the carry and sum. (not accept from a previous addition)



it is a multiple output circuit.

Full Adder

Compute the sum bit and the carry bit when two bits and a carry are added.



A half adder and multiple full adders can be used to produce the sum of n bit integers.